

#1: CS1 Review

1. Write a function add:

- input parameter: two integers n1, n2
- return value: the sum of n1 and n2

```
def add(n1,n2):  
    # ADD ADDITIONAL CODE HERE!  
  
print(add(3,4)) # 7  
print(add(3,5)) # 8
```

2. Write a function printAdd:

- input parameter: two integers n1, n2
- return value: 없음
- action: **print** out the sum of n1 and n2
 - return을 쓰면 안되고 대신 print를 사용
 - 함수에서 아무것도 return 하지 않으면 None이 자동으로 return 됨

```
def printAdd(n1,n2):  
    # ADD ADDITIONAL CODE HERE!  
  
printAdd(3,4)          # 7  
printAdd(3,5)          # 8  
print(printAdd(3,4))   # 7 None
```

3. Write a function maximum:

- input parameter: two integers n1, n2
- return value: the maximum value among n1 and n2

```
def maximum(n1,n2):  
    # ADD ADDITIONAL CODE HERE!  
    # use if-else statement  
  
print(maximum(5,7))    # 7  
print(maximum(7,5))    # 7  
print(maximum(5,5))    # 5
```

4. Write functions `printMultTable1` and `printMultTable2`:

- input parameter / return value: 없음
- action: print out parts of the multiplication table as in the outputs

† copy and slightly modify `printMultTable0`

- the meaning of `for i in range(1, 10, 2):` ?
- the meaning of `for i in range(1, 10, 2):`
`for j in range(1, i+1):` ?

Output of `printMultTable1()`:

```
1 2 3 4 5 6 7 8 9
3 6 9 12 15 18 21 24 27
5 10 15 20 25 30 35 40 45
7 14 21 28 35 42 49 56 63
9 18 27 36 45 54 63 72 81
```

Output of `printMultTable2()`:

```
1
3 6 9
5 10 15 20 25
7 14 21 28 35 42 49
9 18 27 36 45 54 63 72 81
```

5. Write a function `sumSquares`:

- input parameter: an integer list `a` (of length `n`)
- return value: the value of $a[0]^2 + a[1]^2 + a[2]^2 + \dots + a[n-1]^2$
 - i.e. the sum of squares of elements in the list `a`
 - e.g. `sumSquares([4,3,12])` returns 169 ($= 4^2 + 3^2 + 12^2$)
 - recall that the Python code to compute x^2 is `x**2`

```
def sumSquares(a):
    n = len(a)
    sum = 0
    # ADD ADDITIONAL CODE HERE!
    for i in range(n):

print(sumSquares([3,5,4])) # 50
print(sumSquares([2,5,4,0,1,-1,5,1])) # 73
```

6. Write a function square:

- input parameter: a list of integers
- return value: a new list with the same length where each element is squared
 - e.g. `square([1,3,5,6,7])` returns `[1,9,25,36,49]`

```
def square(a):
    n = len(a)
    b = [None] * n      # empty list of length n = len(a)

    # ADD ADDITIONAL CODE HERE!

    return b

L = [3,1,5,2,4]
print(square(L))      # [9,1,25,4,16]
print(L)              # [3,1,5,2,4]
print(square([7,6,3,1,5,8,2,4])) # [49,36,9,1,25,64,4,16]
```

7. Write a function findMin: (min/max pattern)

- input parameter: a list of integers
- return value: the minimum value in the list elements

```
def findMin(a):
    min = a[0]

    # ADD ADDITIONAL CODE HERE!
    for i in range(1,len(a)):
        if a[i] < min:

print(findMin([7,8,3,4,3,6]))      # 3
print(findMin([3,5,7,2,7,2,3,8,6])) # 2
```

8. Write a function countZero: (counter pattern)

- input parameter: an integer list numbers
- return value: the number of occurrences of 0 in numbers

```
def countZero(numbers):
    # ADD ADDITIONAL CODE HERE!

print(countZero([0,4,0,-2,4,0]))      # 3
print(countZero([1,0,-2,4,0,0,-7,0,5])) # 4
```

9. Write functions somePrime and allPrime : (“for some” and “for all” pattern)

- input parameter: a list of positive integers
- return value: a boolean
 - True: somePrime: if there is a prime in the list
allPrime: if all the numbers in the list are primes
 - False: otherwise

```
# "for all" pattern
def isPrime(p):
    if p <= 1: return False
    for i in range(2, p//2+1):
        if p % i == 0:          # not (p % i != 0)
            return ??
    return ??

# "for some" pattern
def somePrime(numbers):
    # ADD ADDITIONAL CODE HERE!
    for i in range(len(numbers)):
        if isPrime(numbers[i]):

# "for all" pattern
def allPrime(numbers):
    # ADD ADDITIONAL CODE HERE!

num1 = [217, 287, 143, 163, 319]
num2 = [217, 287, 143, 169, 319]
num3 = [223, 281, 227, 151, 149]
print(somePrime(num1), allPrime(num1))    # True False
print(somePrime(num2), allPrime(num2))    # False False
print(somePrime(num3), allPrime(num3))    # True True
```

10. Write a function allDistinct: (“for all” pattern)

- input parameter: an integer list numbers
- return value: a boolean
 - True: if all numbers[0], numbers[1], numbers[2], ... are distinct
 - False: otherwise

```
def allDistinct(numbers):  
    # ADD ADDITIONAL CODE HERE!  
    for i in range(len(numbers)):  
        for j in range(i+1, len(numbers)):  
  
print(allDistinct([1,3,2,5,2,1]))    # False  
print(allDistinct([1,0,2,5,3,4]))    # True
```

Write a function allWithinRange: (“for all” pattern)

- input parameter: an integer list numbers, and two integers lower, upper
- return value: a boolean
 - True: if $\text{lower} \leq \text{numbers}[i] \leq \text{upper}$ for all $i = 0, 1, \dots$
 - False: otherwise

```
def allWithinRange(numbers, lower, upper):  
    # ADD ADDITIONAL CODE HERE!  
  
print(allWithinRange([1,0,2,6,3,4], 0,5))    # False  
print(allWithinRange([1,0,2,5,3,4], 0,5))    # True
```

Write a function isPermutation using the functions allDistinct and allWithinRange implemented above: (very simple. you can implement it just in one line!)

- input parameter: an integer list numbers
- return value: $\begin{cases} \text{True} & \text{if the list numbers is a \textbf{permutation}} \\ \text{False} & \text{otherwise} \end{cases}$

† An integer list numbers with length n is called a **permutation** if
all numbers[0], numbers[1], ..., numbers[n-1] are distinct and
 $0 \leq \text{numbers}[i] \leq n-1$ for all $i = 0, 1, \dots, n-1$

```
print(isPermutation([1,3,2,5,2,1]))    # False  
print(isPermutation([1,0,2,5,3,4]))    # True  
print(isPermutation([1,0,2,6,3,4]))    # False
```